P09 Damage and Knockback

CRE132 worksheet

What you'll make

Getting hurt the way action games do it. Walk into a cat or onto the spikes and you lose 1 health, get knocked up and away from the hit, and flash for a second. While you flash, nothing can hurt you, so one touch can't take all your health.

You'll meet **invincibility frames** ("i-frames"), run two **coroutines** at once, and keep the "what hurts" and the "how the player reacts" in separate scripts.

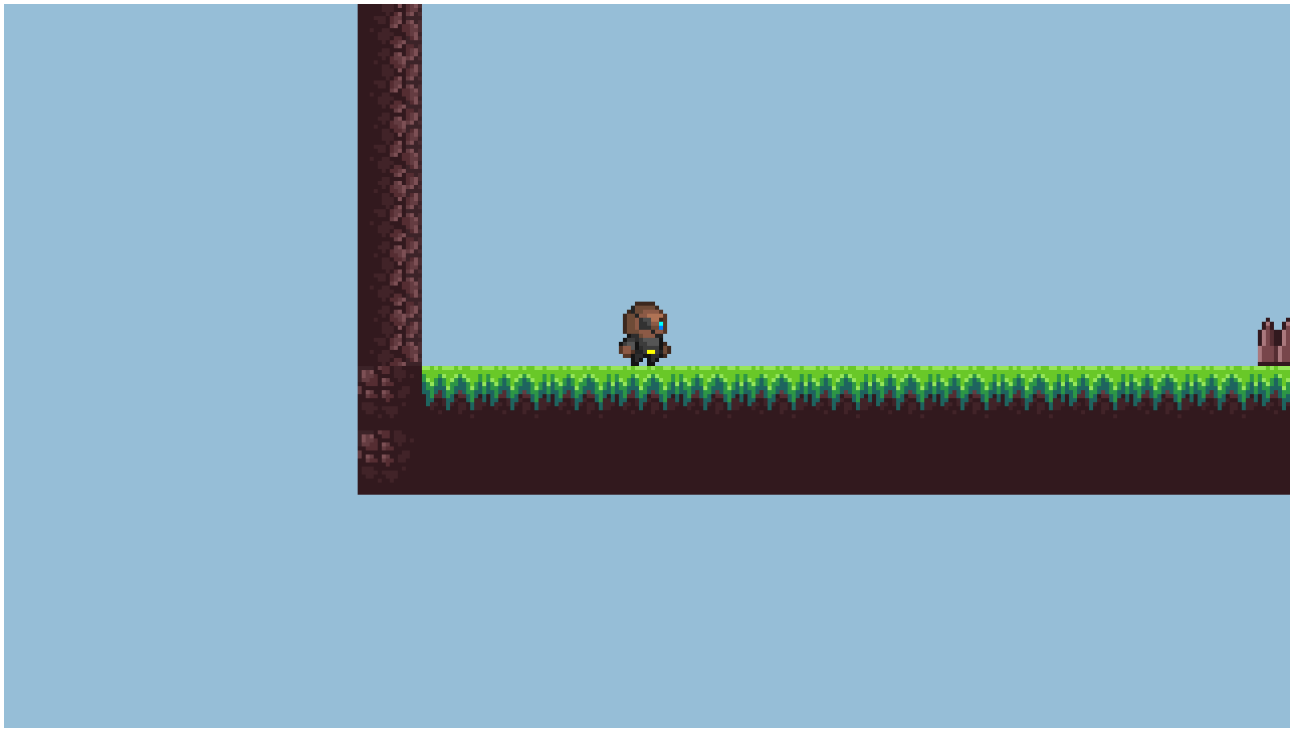


Figure 1: The start of the level: the player by the left wall, with the spikes further along.

Before you start

1. Unzip `CRE132_P09_DamageAndKnockback_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P09_DamageAndKnockback_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P09_DamageAndKnockback`.
3. Press **Play**. You can run (A/D or arrows) and jump (Space). The three cats patrol (P08's `EnemyPatrol`), but touching them or the **Spikes** does nothing yet. The player has **Health** 5. Press Play again to stop.

The problem: if a cat simply took health every physics step while you touched it, you'd lose all 5 health in a tenth of a second. The fix is a short time after each hit when you can't be hurt.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. How the player reacts: PlayerHurt

Make a folder `Assets/_game/Scripts/P09_DamageAndKnockback` and create `PlayerHurt` in it. You write this one first because the script in part 2 calls it.

```
// How the player reacts to being hit: lose health, get knocked back away from the hit,
// then flash for a short time while nothing can hurt again (invincibility frames, or
// "i-frames").
using System.Collections;
using UnityEngine;

public class PlayerHurt : MonoBehaviour
{
    [SerializeField] private Vector2 knockbackVelocity = new Vector2(6f, 8f);
    [SerializeField] private float knockbackTime = 0.25f;
    [SerializeField] private float invincibleTime = 1f;
    [SerializeField] private float flashInterval = 0.1f;

    private Health health;
    private Rigidbody2D body;
    private SpriteRenderer spriteRenderer;
    private BetterJumpController controller;
    private bool isInvincible;

    private void Awake()
    {
        health = GetComponent<Health>();
        body = GetComponent<Rigidbody2D>();
        spriteRenderer = GetComponent<SpriteRenderer>();
        controller = GetComponent<BetterJumpController>();
    }

    // Called by anything that hurts the player, with the point the hit came from.
    public void Hurt(int damage, Vector2 hitFrom)
    {
        if (isInvincible)
        {
            return;
        }

        health.TakeDamage(damage);
        StartCoroutine(Knockback(hitFrom));
        StartCoroutine(Invincibility());
    }

    private IEnumerator Knockback(Vector2 hitFrom)
    {
        // Away from the hit: left if it came from the right, right if it came from the
        // left.
        float direction = 1f;
        if (hitFrom.x > transform.position.x)
        {
            direction = -1f;
        }

        // Switch the controller off so it doesn't overwrite the knockback with the
        // player's input.
        controller.enabled = false;
        body.linearVelocity = new Vector2(direction * knockbackVelocity.x,
            knockbackVelocity.y);
```

```
        yield return new WaitForSeconds(knockbackTime);

        controller.enabled = true;
    }

    private IEnumerator Invincibility()
    {
        isInvincible = true;

        // Flash: hide and show the sprite every flashInterval seconds.
        float timer = 0f;
        while (timer < invincibleTime)
        {
            spriteRenderer.enabled = !spriteRenderer.enabled;
            yield return new WaitForSeconds(flashInterval);
            timer = timer + flashInterval;
        }

        spriteRenderer.enabled = true;
        isInvincible = false;
    }
}
```

What's going on:

- **`Hurt` is public**: anything harmful calls it with the damage and the point the hit came from. If `isInvincible` is true it returns straight away, so repeated touches do nothing.
- **Two coroutines start at once.** `Knockback` is short (0.25 s) and `Invincibility` is longer (1 s). Each one pauses at its own `yield return new WaitForSeconds(...)` while the game carries on.
- **Knockback switches the controller off.** The controller sets the velocity from your input every step, so it would wipe out the knockback straight away. It's the same trick as P03's dash.
- **The flash** turns the Sprite Renderer off and on every `flashInterval` seconds. At the end it makes sure the sprite is visible again before ending the invincibility.

Add it: select the **Player** and **Add Component > Player Hurt**. Leave the values: **Knockback Velocity** X 6, Y 8, **Knockback Time** 0.25, **Invincible Time** 1, **Flash Interval** 0.1.

Test it: press Play. Nothing new happens yet, but the Console should have no red errors.

2. Things that hurt: HurtsPlayer

Create `HurtsPlayer` in the same folder:

```
// Put on anything that hurts the player by touching it: enemies (solid colliders) or
// spikes (triggers).
// It hurts on every physics step of contact, but the player's i-frames stop it
// hurting again straight away.
using UnityEngine;

public class HurtsPlayer : MonoBehaviour
{
    [SerializeField] private int damage = 1;

    private Collider2D myCollider;

    private void Awake()
    {
```

```
        myCollider = GetComponent<Collider2D>();
    }

    private void OnCollisionStay2D(Collision2D collision)
    {
        TryHurt(collision.gameObject);
    }

    private void OnTriggerStay2D(Collider2D other)
    {
        TryHurt(other.gameObject);
    }

    private void TryHurt(GameObject other)
    {
        PlayerHurt player = other.GetComponent<PlayerHurt>();
        if (player != null)
        {
            // The closest point of this collider to the player, so a long row of spikes
            // still knocks the player away from where they actually touched it.
            Vector2 hitFrom = myCollider.ClosestPoint(other.transform.position);
            player.Hurt(damage, hitFrom);
        }
    }
}
```

- **Two "Stay" methods, one job.** Cats have solid colliders, so they get `OnCollisionStay2D`. The spikes are a trigger, so they get `OnTriggerStay2D`. Both call `TryHurt`, so the script works on either.
- **`GetComponent<PlayerHurt>()`** finds out whether the thing touching is the player. Anything else (the ground, another cat) has no PlayerHurt, so it's skipped.
- **`ClosestPoint`** gives the nearest point of this collider to the player. For a long strip of spikes, the middle of the strip could be far away, so this point is what decides which way to knock the player.
- **HurtsPlayer doesn't care how the player reacts.** All the reaction lives in PlayerHurt.

Add it: select the three **Cat** objects (in **Enemies**) and add **Hurts Player**. Then select **Spikes** (inside **Grid**) and add **Hurts Player** to it too. Its Tilemap Collider 2D already has **Is Trigger** ticked. Leave **Damage** at 1 on all of them.

Test it: walk into a cat. You lose 1 health, fly up and away from it, and flash for a second. Do the same on the spikes. Lose all 5 health and the level restarts.

Check your work

- ☐ Touching a cat or the spikes takes 1 health and knocks you away from the hit.
- ☐ You flash for about a second, and nothing can hurt you while you flash.
- ☐ Hit from the left, you fly right; hit from the right, you fly left.
- ☐ You can move again a quarter of a second after the hit.
- ☐ Compare with `CRE132_P09_DamageAndKnockback_Finished` if something behaves differently.

Key points

- **I-frames** are standard in action games: they stop "damage every frame" and give the player a moment to escape.
- **Two coroutines can run at once**: knockback (short) and invincibility (longer).
- **Keep the cause and the reaction apart.** HurtsPlayer only says "you were hit, from here". PlayerHurt decides what that means, so new hazards need no new code.

If something goes wrong

- **“The type or namespace name ‘PlayerHurt’ could not be found”:** PlayerHurt has an error, or its file and class names don’t match. Fix the first red error first.
- **NullReferenceException in PlayerHurt:** PlayerHurt must be on the **Player**, next to its Health, Rigidbody 2D, Sprite Renderer and Better Jump Controller.
- **The cats hurt you but the spikes don’t:** HurtsPlayer is missing from **Spikes**, or its collider’s **Is Trigger** has been unticked.
- **The knockback barely moves you:** the controller is still on. Check `controller.enabled = false;` comes before the velocity line.
- **You stay invisible after a hit:** the line `spriteRenderer.enabled = true;` after the loop is missing.

Try it

1. Raise **Knockback Velocity** for a bigger bounce, or set it to (0, 0) to feel the difference.
2. Let the player pass through enemies while invincible: put the cats on their own layer and use `Physics2D.IgnoreLayerCollision`.
3. Play S05’s hurt sound in `Hurt`.